LeetCode 经典面试题 150 解题思路总结

1. 合并两个有序数组

方法一：双指针（正向排序），i 指针和 j 指针分别指向两个数组的 0 下标，每次比较两个指针指向的元素，指向较小的元素的指针往后移，直到某个数组的下标超出数组 len。

方法二：双指针（逆向排序）,由于正向排序需要辅助数组，可以采用从后往前比较。

2. 移除指定的元素（val)

方法：采用双指针；i 指针指向需保留元素的插入位置，j 指针作为遍历指针遍历数组，如果遍历的元素不等于 val，就把当前元素插入到 i 指针记录的位置，i 指针后移。

3.删除有序数组的重复项|

方法：双指针；i 指针指向已经插入的最后一个元素的位置（初始为 0，表示已插入 1 个元素），j 指针作为遍历指针（初始为 1），如果遍历的元素不等于 i 指针指向的元素，就把当前元素插入到 i 指针的后一个位置。

4.删除有序数组的重复项||

方法：双指针；判断方法区别：如果当前遍历元素不等于已插入元素的最后一个元素，直接插入；反之继续判断当前元素是否不等于插入的倒数第二个元素（或已插入元素只有一个），不等于则插入元素，否则不插入。

5. 多数元素（找到数量大于一半的元素）

方法：计数法；如果把众数表示+1，其他元素表示-1，那么数组总和一定大于 0。把众数初始化为第一个元素，count 设为 1，如果后续遍历的元素等于众数，count++，反之 count--；如果 count==0，则设当前遍历元素作为众数，count=1，遍历完数组即可找到众数。

6. 轮转数组（将数组元素向右轮转 k 次）

方法一：环状替换;

证明：设坐标为 x，数组长度为 n，轮转后的位置为(x+k)%n，下一次轮转后坐标为(x+2k)%n，...，设经过 b 次轮转后回到初始位置，则有（x+bk)%n=x，则有 bk=an（a 表示轮转的环圈数），为使轮转圈数尽可能小，an 应该尽可能小，即为 n,k 的最小公倍数，记为 LCM。每次轮转环的元素个数为 LCM/k，数组元素个数为 n，故共需要轮转的次数为：n /(LCM/K)=nk/LCM，即为 n，k 的最大公约数，记为 GCD。

实现：经过 GCD 次循环（坐标 x=0,...,GCD-1），每次循环中：将坐标为 x 的元素替换到 next=（x+k）%n 的位置，再轮转 next 的坐标，直到 next 的坐标等于 x 时，进入下一次循环。

方法二：三次翻转数组

实现：设数组的元素为（a1,a2,.....,an)；

第一次翻转：（ak-1, ak-2, . . . , a1, ak, ak+1, . . . an)

第二次翻转：（ak-1, ak-2, ... , a1, an, an-1, . . . , ak)

第三次翻转：（ak, ak+1, . . . an, a1, a2, . . ak-1)

7. 买股票的最佳时机（只能购买一支股票）

方法：记录前 i-1 天的最低购买股票价格 MinPrice，则第 i 天的售卖股票的最大利润记为

prices[i]-MinPrice，所以在遍历数组时不断的更新最低股票价格，即可算出当天售出股票的最大利润，同时记录全局最大利润即可。

8. 买股票的最佳时机（任意时刻只能持有一只股票）
方法一：贪心算法。只要某一支股票后续能够给我们提供利润即买入，考虑第 i 天的股票价格，如果下一天的股票价格更高，则第 i 天的股票可以给我们提供利润，但如果下一天的价格更低，则无需购买第 i 天的股票，因为如果后续出现更高价格时的股票，第 i+1 天的股票可以提供的利润更多。

方法二：动态规划。定义状态 dp[i][0]和 dp[i][1];dp[i][0]表示第 i 天不持有股票时的最大利润，dp[i][1]表示第 i 天持有股票时的最大利润。考虑 dp[i][0]，若第 i-1 天不持有股票，则 dp[i][0]=dp[i-1][0],若第 i-1 天持有股票，dp[i][0]=dp[i][i-1]+prices[i]; 考虑 dp[i][1],若第 i-1 天持有股票，则 dp[i][1]=dp[i-1][1],若第 i-1 天不持有股票，dp[i][1]=dp[i][0]-prices[i]；初始状态 dp[0][0]=0,dp[0][1]=-prices[0]。

则状态转移方程：$\begin{cases} dp[i][0] = \max(dp[i-1][0], dp[i-1][1] + prices[i]) \\ dp[i][1] = \max(dp[i-1][1], dp[i-1][0] - prices[i]) \end{cases}$

优化：由于当前状态只与前一步状态有关，可以用 Pre0 和 Pre1 记录前一天两种情况下的最大利润。

9. 跳跃游戏（是否能到达最后一个下标）
方法：贪心思想。实时维护一个可以跳跃到的最大下标 CanIndex,初始为 0；
则 CanIndex=Max(CanIndex， i+nums[i]),其中 i=0,1,...,CanIndex。通过循环不断更新 CanIndex 的值，判断是否能等于或大于数组长度-1。

10. 跳跃游戏 2（到达最后一个下标的最少次数）
方法：贪心思想。
方式一：（逆向）为了最少次数到达最后的下标，我们逆向思考，能一步到达最后下标的上一步落点有哪些下标，且为了最小次数，我们应选择能够到达最后下标中的上一步落点中的最小下标；然后寻找能一步到达倒数第二步下标的落点中的最小下标.....
方式二：（正向）从下标 0 出发，初始能到达的最远位置记为 border=0，只要下标小于等于 border 的位置我们都可以一步到达，同时记录 next_border=max(next_border,i+nums[i])，遍历到 border 后，如果想走的更远就需要增加步数，即 step+1，同时将 border 更新为 next_border,后续继续维护 next_border,直到达到最后一个下标。

11. H 指数
方法一：计数排序。其思想是数组元素的范围是确定的。计数数组 counter[i]表示数组中等于 i 的元素个数有多少个，则统计小于 i（或者大于 i）的元素个数可以使用累加法统计，即可用于排序。

方法二：二分排序。我们需要这样找到一个 h，使得它是满足性质（至少有 h 篇论文的引用次数至少为 h）的最大值，使得小于等于 h 的所有 x 都满足这个性质。我们可以不断采用二分法不断去逼近，左右边界 low=0，high=nums.length;如果当 mid 满足这个性质（至少有 mid 篇的引用次数至少为 mid），则令 low=mid,不满足这个性质时，令 high=mid-1；直至循环条件不满足（low<high)，此时 low 即为我们寻找的 h。

12. O(1)时间插入、删除和获取随机元素（等概率）

方法：哈希表+可变长数组。由于需要满足 O（1）获取随机元素这个性质，考虑可变长数组作为存储容器，O（1）实现不重复插入和删除，则需要通过哈希表实现，同时 value 记录元素的下标。注意的是数组的删除操作是 O(n),但是没有要求保持稳定性，可以选择将最后的数组元素与删除元素交换位置（通过哈希表 O(1)查到下标）,然后实现 O（1）删除。

13. 除了自身以外数组的乘积

方法:用数组 L[i]和 R[i]通过两次遍历,分别记录下标为 i 的元素的左边全部元素和右边全部元素的乘积，然后 L[i]和 R[i]的乘积即为目标 mul[i]。

优化：可以用单个变量 value 记录右边元素的乘积，这样在遍历右边全部元素乘积的过程中即可统计 mul[i]，这样只需要一个数组（返回数组）。

14. 加油站

方法：贪心算法。假设从加油站 x 出发，最远可以到达加油站 y。则根据题目则有

$$\sum_{\mathrm{i=x}}^{y} gas[i] < \sum_{i=x}^{y} \mathrm{cost[i]}, \qquad \sum_{i=x}^{j} gas[i] \geq \sum_{i=x}^{j} \cos t[i]，其中\ j \in [x, y)$$

假设我们可以在加油站 x 和 y 之间找到一个加油站 z，从加油站 z 出发可以到达比加油站 y 更远，但

$$\sum_{i=z}^{y} gas[i] = \sum_{i=x}^{y} gas[i] - \sum_{i=x}^{z-1} gas[i]$$

$$> \sum_{i=x}^{y} \cos t[i] - \sum_{i=x}^{z-1} gas[i]$$

$$> \sum_{i=x}^{y} \cos t[i] - \sum_{i=x}^{z-1} \cos t[i] = \sum_{i=z}^{y} \cos t[i]$$

故不可能在 x 和 y 之间找到一个加油站走的更远,故下一次应该从到达最远加油站 y 开始继续往后尝试。代码优化：从下标 start=0 的加油站开始尝试，如果最远到达的加油站的坐标 end 小于遍历的初始坐标 start，则说明无法到达（因为小于 start 的坐标已经尝试过），可以通过 start<len 作为跳出循环条件。

15. 分发糖果

方法一：贪心思想。我们尝试把题目的要求（相邻的孩子中，分数较高的孩子获得更高的糖果）化分为满足左规则（ratings[i]>ratings[i-1]）和右规则（ratings[i]>ratings[i+1]）;以左规则为例，如果下标为 i 孩子的分数大于下标 i-1 的孩子的分数，则糖果数 Candy_left[i]=Candy_left[i-1]+1，否则 Candy_left[i]=1，这样下来所有的孩子能在最少糖果总数下满足左规则要求；同理可以求得满足右规则要求每个孩子的 Candy_right[i]，我们取每个孩子满足左规则和右规则的糖果数的较大者，这样即可满足题目要求。

方法二：贪心思想。我们可以注意到若要最少糖果数满足题目要求，当 ratings[i]>ratings[i-1],则 Candy[i]=Candy[i-1]+1 时可以最少糖果总数满足；当 ratings[i]<ratings[i-1],我们令 Candy[i]=1 可以达到总数最少，但可能出现一种异常情况，Candy[i-1]=1 导致不满足题目要求，故我们需要把 Candy[i-1]+1，若后续又来一个更

小值，则前两个都需要+1，因此可以考虑记录单调递减序列的长度 dec，每次总糖果增加 dec 即可，但是当单调序列中的第一个孩子的糖果数等于前一个孩子的糖果数（此时不满足题目要求了），需要把前一个孩子也考虑到单调递减序列中（即 dec++）。所以可以记录一个单调递增序列的长度 inc(也表示单调递减序列中第一个孩子前一个孩子的糖果数）。

16. 接雨水

方法一：单调栈。分析题目，我们可以知道当形成类似这样的（v）的坑时，即可以接到水；因此我们可以考虑维护一个单调递减栈（记录柱子的下标），这样栈中元素对应的柱子高度相当于已经形成坑的左侧（栈中的元素还有靠近栈底的邻居），这样当我们遇到比栈顶下标对应的柱子高度更高的柱子时，即可形成一个接水的坑。记遍历到的下标为 index，栈顶的下标为 top，次栈顶的下标为 minor（如果存在），则接到的水量

water=（index-minor-1）*（min(height[minor],height[index])-height[top])

方法二：动态规划。对于每个下标为 i 的柱子而言，如果我们能知道其左边的最高柱子高度 left_max（涵盖该柱子）,且也能知道其右边的最高柱子高度 right_max（涵盖该柱子），则柱子能够接到的水量为 min(left_max,right_max)-柱子 height；故我们可以用两个数组 left_max[i],right_max[i]记录柱子 i 左边的最高柱子高度和右边的最高柱子高度;其中状态转移方程可化为

$$\text{left_max}[i] = \max(left_\max[i-1], height[i]),\ \ i > 0$$
$$right_\max[i] = \max(right_\max[i+1], height[i],\ \ i < n-1$$

方法三：双指针。假设有两个指针 left=0、right=n-1，left_max 维护 left 左边的最高柱子高度（涵盖该柱子），right_max 维护 right 右边的最高柱子高度（涵盖该柱子）；如果 left_max<right_max，则对于下标 left 的柱子而言，无论后面在其右侧出现比 right_max 更高的柱子出现时，都被 min(left_max,right_max)这个条件排除在外了，即下标为 left 的柱子接水量已经确定，即可令 left++继续遍历；如果 left_max>right_max 时，对于下标为 right 的柱子而言，无论后面在其左侧出现比 left_max 更高的柱子出现时，也被 min(left_max,right_max)这个条件排除在外，故下标为 right 的柱子接水量已经确定，令 right--继续遍历。

17. 罗马数字转整数

方法：哈希表。通过建立罗马数字到整数的哈希表 map，O(1)获取罗马数字的对应数值；根据规则罗马数字中小的数字在大的数字的右边,当遍历时遇到下标 i 的数值小于 i+1 的数值时，即出现特殊表示法。

18. 整数转罗马数字

方法一：哈希表。通过建立数字到罗马数字的哈希表 map，再依次计算 num 每一位的数字和此时的基 ratio（ratio 记为小于 num 的十的指数，1999 的指数为 1000），即可结合数位和 ratio 得到此数位的罗马数字。

19. 最后一个单词的长度

方法：方向统计。只要当前字符是空（‘ ’）,即指针不断减小，直到出现一个非空字符指针不断减小计数 count（当前字符非空），当前字符为空字符则计数停止返回 count。

正向统计。只要当前字符是空字符时，不断指针增加。出现非空字符时，统计 count 重置计数，直到指针超出字符长度停止，返回 count.

20. 最长公共前缀

方法一：横向判别。所有字符串的最长公共前缀也是任意两个字符串的公共前缀的前缀，且长度不可能超过该两个字符串的最长公共前缀长度。故我们可以依次求出前两个字符串的最长公共前缀 $prefix_{1-2}$，用 $prefix_{1-2}$ 和第三个字符串去求前三个字符的最长 $prefix_{1-3}$，这样依次遍历即可求得 $prefix_{1-n}$。

方法二：众向判别。依次判断所有字符串下标为 i=0 的字符是否相同，如果全部相同则 prefix 长度加一，继续判断所有字符串坐标为 i+1 的字符，如果不全相同，则范围（0，prefix）的字串。

21. 反转字符串中的单词

方法：我们可以通过两个指针 start 和 end 轻松找到一个单词的开始和结尾（不包含），流程：循环判断当前字符是否为空字符，是则 start++（循环结束 start 为第一个字符的首字母）；令 end=start，再循环判断当前字符是否为非空字符，是则 end++（循环结束下来 end 为单词的最后一个字符的后一个下标），通过方法 substring（str,startIndex,endIndex）（左闭右开），继续找后续的单词，最后利用 StringBuilder 拼接所有单词即可。

22. Z 字形变换

方法：通过观察题目，可以发现单词的当前字母被添加到了第 i 行，且行索引 i 从 0 逐渐自增到 numRows 后，又逐渐自减到 0，循环往复。故我们可以创建 numRows 长度 StringBuilder 数组，把当前遍历的字符加入到 StringBuilder[i]中；这里巧妙借助一个 flag，初始为 flag=-1，当（i==0||i==numRows-1）flag=-flag；之后每次让 i+=flag；实现巧妙的 N 形循环。

23. 找到字符串中第一个匹配项的下标

方法一：暴力匹配算法。令母串初始下标 start=end=0，从 start 开始逐一字符和匹配串匹配，匹配成功则令 end++，继续和匹配串当前字符进行比较；如果某个字符匹配失效，则令 start++，重复上面操作。

方法二：KMP 算法。暴力匹配算法时间复杂度高的点在于每次匹配失效时，母串的指针都会回溯，而当匹配失效时，匹配失效字符前正确匹配成功的字符是已知的，KMP 方法的巧妙之处是利用匹配正确的字符信息使得母串指针无需回溯。即问题变成：当匹配串的第 i 个字符匹配失效时，我们应该用匹配串哪一个字符去匹配。

例：　　母串：aaaabc　　匹配串：aaabc

当母串下标 i=3 的字符 a 匹配失效时（a!=b),尝试利用其已知信息。我们右移匹配串观察，发现需要匹配的字符应为匹配正确字串的最大前后缀公共长度的后一个字符。

a	a	a	a	b	c
	a	a	a	b	c

设 next[i]数组为：当第 i 个字符匹配时，匹配串应该用哪个索引的字符去继续匹配。通过上面分析，可知 next[i]为索引 0,...,i-1 的子串的最大公共前后缀长度 len +1;

那如何求 next[i]?

令 next[0]=-1,表示第一个字符匹配失效时母串指针后移；next[1]=0（匹配第一个字符）当 i>=2 时，由于我们已知 next[j](j<i),设索引 0,...,i-2 的子串的最大公共前后缀长度为 N（N<i)，next[i-1]=N（索引）。

$$A_1 A_2 A_3 \ldots A_N A_{N+1} \ldots A_{i-N-1} A_{i-N} \ldots A_{i-2} A_{i-1} A_i \ldots$$

即我们已经知道子串 $A_1 A_2 A_3 \ldots A_N$ 与子串 $A_{i-N-1} A_{i-N} \ldots A_{i-2}$ 相等，则若 A_{N+1} 与 A_{i-1} 相等，即下标为 i 的最大公共前后缀长度为 N+1；若 A_{N+1} 与 A_{i-1} 不相等，相当于字符串 $A_1 A_2 A_3 \ldots A_N A_{N+1}$ 第 N+1 个字符匹配失效（A_{N+1} != A_{i-1})时，应用哪个索引的字符去匹配，而 next[N]我们是已知的，则可循环去判断。算法的流程如下：

```
next[0]=-1,next[1]=0;
for 循环( i=2;i++){
    j=next[i-1];
    While(j>=0&&str[j]!=str[i-1]) j=next[j];
    next[i]=j+1;
    }
```

24. 文本左右对齐

方法：贪心思想。观察题目可以容易知道：每个单词间至少需要隔一个空字符。如何统计每行最多可以放下多少个单词呢？我们设每行最多可以放下 n 个单词 $word_1$，$word_2$, ...，$word_n$，则该行的长度为 n 个 word 的长度(记为 sum)和与 n-1 个空白字符的长度；通过循环执行步骤：记起始坐标为 left=right=0，sum=0 如果 len(word[right])+sum+right-left<maxWidth 时，即可添加新单词 word[right]去，则令 right++，sum+=len(word[right]);可以很容易计算每行最多可以存放几个字符。再依次考虑最后一行（例外）和其他行空格的插入数量即可。

25. 验证回文串（移除所有非字母数字字符，并转小写之后判断是否为回文串）

方法：双指针。我们用指针 left 和 right 分别指向原串的起始字符和结束字符，通过循环：{如果当前字符不是字母数字字符，left++}，可以找到左侧的第一个字母数字字符，再通过循环：{如果当前字符不是字母数字字符，right--}，可以找到右侧的第一个字母数字字符，然后判断是否相等，相等即继续重复上述操作（直到 left>=right 时即为回文串），不相等即不为回文串。

26. 判断子序列（判断 s 是否为 t 的子序列）

方法：双指针。假设 s 的某个字符 a 在 t 中有多个匹配位置 pos_1,pos_2,...;且位置 pos_i（i>1)后可以依次在 t 中找到 s 的剩余字符，显然字符 a 选择第一个位置 pos_1 也可以满足在 t 中依次找到；反之如果字符 a 选择 pos_1 位置作为匹配位，后续无法在 t 中依次找到 s 剩余字符的匹配位，那么选择 pos_i (i>1)也是无法找到的，因为能匹配的区间更小。因此我们只需依次找 s 中每个字符在 t 中的第一次出现位置，流程如下：设 s_index=0,t_index=0；如果 t 中索引 t_index 的字符不等于 s 中索引 s_index 的字符，t_index++;如果出现相等时，s_index++；直到 s 全部字符匹配成功。

优化：上述方法的执行流程是依次遍历寻找下一个字符的匹配位置，我们可以提前预处理字符串 t，方便我们快速在 t 中找到某个字符的匹配位置。设 pos[i][j]表示在字符串 t 中索引 i（包含 i)以后字符 j 出现的第一次坐标；(0<=j<=25 , 0<=i<=len(t))

若 t 中索引 i 的字符为 j，则 pos[i][j]=i；若 t 中索引 i 的字符为 j，那 pos[i][j]=pos[i+1][j];那么状态转移方程为：

$\mathrm{dp}[i][j]=\begin{cases} i, t[i]=j \\ dp[i+1][j], t[i]!=j \end{cases}$，边界状态 $\mathrm{d}p[len(t)][j]=-1, j=0,1,...,25$,-1 表示后续没有匹配位，因此只需逆序遍历字符 t 即可。

27. 两数之和 II——输入有序数组

方法：双指针。令指针 left=0，right=nums.length-1；循环执行：如果 nums[left]和 nums[right]的和 sum 小于 target，则 left++（因为如果减少 right，sum 只会变得更小远离 target）；大于 target，令 right--（因为增加 left，sum 只会变得更大远离 target）；等于 target 结束循环即可。

28. 盛最多水的容器

方法：双指针。设容器的左右边界的索引分别为 left，right，根据题目描述容器的盛水量 water=min(height[left],height[right])*(right-left)；假设初始 left=0,right=length-1,无论增加 left 亦或减少 right，right-left 的值是一样的；所以我们选择增加 left 还是减少 right 取决于 height[left]和 height[right]的值；应当优先移动 height 较小的那个索引；假如 height[left]小于 height[right]，减少 right 是无法使 min(height[left],height[right])的值变大的（至少为 height[left]）。

29. 三数之和（原始数组不有序，计算不同索引的三个数的和为 0 的所有去重组合）

方法：双指针。计算不同索引的三个数的和为 0，可以选择先固定一个数的索引为 i，问题转换为找到不同索引的两个数之和为-nums[i]，如果数组不是有序的，则找到目标为-nums[i]的两个数的复杂度为 $O(n^2)$；如果数组是有序的，则通过双指针复杂度可以降为 O(n)，加上第一个数索引 i 的循环，整体复杂度可以降为 $O(n^2)$;步骤：

```
外层循环（索引 i）{
    //去重相同的 nums[i]
    target=-nums[i]，j=i+1,k=nums.length-1;
    内层循环（j<k){
        If(nums[j]+nums[k]>tar)k--;
        If(nums[j]+nums[k]<tar)j++;
        Else  结果集之一
        //去重相同的 nums[j]和 nums[k]
        }
```

Nums[i]的去重：while 循环(i>0&&nums[i-1]==nums[i])i++;

Nums[j]和 Nums[k]的去重：用 pre 记录当前值，如果 nums[j]==pre,j++（nums[k]==pre,pre--）

30. 长度最小的子数组

方法：滑动窗口。根据题目要求，用两个指针标记为窗口的开始 start 和末尾 end，同时变量 cnt 记录满足条件最小的子数组长度，如果窗口内所有元素之和 sum 小于目标值 target，那我们可以考虑扩充窗口（end++），如果元素之和 sum 大于等于 target，此时是一个满足的子数组，用子数组长度 len 和 cnt 比较，如果 len 更小赋值给 cnt 即可，然后收缩窗口（start++），如果元素之和 sum 不满足就扩充，满足就继续收缩。

31. 无重复字符的最长字串

方法：滑动窗口和哈希表。根据题目要求，用两个指针 left=0,right=0 记录窗口的开始索引和结束索引，用哈希表记录窗口内的字符，O(1)判断当前字符是否已在窗口中，同时用 cnt

记录最长字串的长度；流程：如果当前字符 ch 不在哈希表中，则选择扩充窗口（right++)，同步更新 cnt 的值；如果字符 ch 已在哈希表中，则不断收缩窗口（left++）直到索引 left 为上次出现的字符 ch 的下一个索引。

伪代码：while 循环（strs[left]!=ch){

```
left++;
Hashset.remove(strs[left]);
}
left++;    //left 指向上次出现的字符 ch 的下一个索引
```

32. 串联所有单词的字串（定长滑动窗口）

方法：滑动窗口+哈希表。根据题目要求，我们需要在母串 s 中找到以单词组 words 任意顺序拼接的子串（n 为 words 数组的长度，m 为 word 的长度）；由于所有单词的长度相等，即目标子串的长度是固定的（m*n），则我们可以去维护一个定长窗口，判断定长窗口内的所有单词是否为给定单词 word。故用哈希表 target 记录 words 的单词数量，用哈希表 current 记录定长窗口内的单词数量。注意到，由于单词 word 的长度是固定的，所以将母串划分为长度为 m 的单词的方法只有 m 种；

$$word_{0\sim m-1}, word_{m\sim 2m-1}, word_{2m\sim 3m-1}\cdots$$

$$word_{1\sim m}, word_{m+1\sim 2m+1}, word_{3m+1\sim 4m}\cdots$$

…

$$word_{m-1\sim 2m-2}, word_{2m-1\sim 3m-2}, word_{3m-1\sim 4m-2},\cdots$$

故只需要 m-1 次循环，分别以 0,1,...m-1 作为窗口的左边界 left，右边界 right=left+m*n，每次移动窗口时移动单词长度 m 即可。

窗口滑动过程：

定长窗口的巧妙：用 overload 记录窗口内 current 中单词的数量超过 target 中单词的数量的超载单词数。

① 新单词进；如果单词 Inword（right 为结尾的单词）在 current 的数量已经等于 target 的数量，令 overload++，当前单词的数量超载;当前单词进入 current 哈希表中。

current.merge(Inword,1,Integer::sum)

② 更新结果；如果 overload 等于 0，表示没有超载的单词，结果记录窗口左边界 left。注意：需要达到定长窗口时才开始记录。

③ 旧单词出；从 current 中移除当前单词 Outword（left 开始的单词）；cur.merge(Outword,-1,Integer::merge);如果移除单词后，current 中的单词数量等于 target 的数量，overload--，该单词不再超载。

33. 最小覆盖子串

方法：滑动窗口+哈希表。根据题目，可以用两个哈希表分别记录目标串的字符数量和滑动窗口内的字符数量，用 matchs 记录窗口内数量已满足目标子串的字符数，用 target 记录目标串的不同字符个数，用 left 和 right 记录窗口的起始和结束，min_left 和 min_right 记录最小覆盖子串的开始和末尾；流程如下：不断扩充窗口，维护窗口内的各字符的数量，当新增字符的数量满足目标串的该字符数量，令 match++；如果 match 等于 target 时，此时窗口子串已经满足要求，维护 min_left 和 min_right；循环收缩窗口直到 match 不等于 target，收缩期间同时维护 min_left 和 min_right;

收缩伪代码： while(match==target){

```
if(min_right-min_left>right-left){
    min_left=left;min_right=right;
    }
    窗口哈希表该字符数量-1；
    if(窗口哈希表该字符数等于目标哈希表该字符数-1)match--;
    }
```

优化：可以用两个字符数组代替哈希表记录字符的数量，访问速度更快。

34. 有效的数独

方法：哈希或模拟。根据题目要求，需要任何行、任何列、任何九宫格中不能有重复的数字，则可以用哈希表记录行、列、九宫格已出现的数字（或者循环判断亦可），borad[i][j]所属的九宫格为$\frac{i}{3},\frac{j}{3}$(下取整）。

优化：可以用数组代替哈希表，因为数字的范围是已知的。row[9][9]记录每行出现过的数字,col[9][9]记录每列出现的数字,submatrix[3][3][9]记录每个九宫格出现的数字。

35. 螺旋矩阵

方法：模拟。通过观察螺旋矩阵的输出顺序，我们用四个变量 t，b，l，r 记录还未行走过的四个边界（上下左右），起始索引 i=0,j=0；步骤：1.列增输出走过的元素，直到到达右边界 r，同时设置步骤 2 的起点； 2.行增输出走过的数字，直到到达下边界 b，设置步骤 4 的起点；3.更新边界，t++，r--，同时判断是否已结束（l>r || t>b）,没结束执行后续步骤； 4.列减输出走过的元素，直到到达左边界 l，同时设置步骤 5 的起点；5.行减输出走过的元素，直到到达上边界 t，同时设置步骤 1 的起点；6.更新边界，t--，l++，同时判断是否结束（l>r || t>b），没结束从步骤 1 开始重复。

36. 旋转图像（顺时针旋转 90）

方法：先转置矩阵，再逆置每行的元素（观察发现）；数组原地逆置的方法：双指针，用两个指针 left=0，right=len-1，循环交换索引 left 和 right 两个元素的值，left++,right--，直到条件 left<right 不满足。

37. 矩阵置零

方法：哈希表。我们可以遍历数组的过程中，用两个哈希表记录哪些行和哪些列的元素需要被置零。（matrix[i][j]==0 代表第 i 行和第 j 列的元素都需要置 0） 空间复杂度：O(m+n)

原地置零：我们发现当第 matrix[i][j]等于 0 时，其 matrix[0][j]（第 0 行）和 matrix[i][0]（第 0 列）两个位置的元素最终都会被置 0，也就是我们可以用原数组的第 0 行和第 0 列作为标记（等同于哈希表），如果第 0 行中的元素 matrix[0][j]等于 0 时，我们令第 j 列的全部元素置零即可，如果第 0 列中的元素 matrix[i][0]的元素等于 0 时，我们令第 i 行的全部元素置零即可；所以我们在遍历数组的过程中，只需维护标记第一行和第一列的元素，即 matrix[i][j]等于 0 时，令 matrix[0][j]和 matrix[i][0]为 0；当我们发现 matrix[0][0]为 0 时产生歧义，无法判断是第一行还是第一列中的元素导致，所以可以两个变量 ZeroFirstRow 和 ZeroFirstCol 去为单独记录第一行和第一列是否包含零即可。

38. 生命游戏

方法：模拟。根据题目，我们可以统计当前位置附近的活细胞数量来更新当前位置的值（即判断是否存活还是死亡），但是我们发现当状态变更时，即 0 变成 1，1 变成 0，会影响统计后续位置附近的活细胞个数，所以简单的思路就是用一个辅助数组来记录更新数组前的状态，会产生 O(m*n)的空间复杂度。假设活细胞的个数为 count，我们可以得到下列规则：

$$\begin{cases} \text{count} < 2 \,||\, count > 3, \text{赋值}0\text{（活细胞死亡）} \\ \text{count} = 2, \text{值不变（活细胞存活，死细胞仍然死亡）} \\ \text{count} == 3\text{，赋值}1\text{（死细胞复活，活细胞仍然存活）} \end{cases}$$

原地记录更新前状态：我们可以用特殊值-1 表示原来的活细胞 1 状态变为死细胞 0；用特殊值-2 表示原来的死细胞 0 状态变为活细胞 1；所以在统计附近活细胞的个数 count 时我们需要同时统计 1 和-1 的值。最后再遍历一边数组修改特殊值为正常值即可。

39. 赎金信

方法：哈希表。根据题目所述，可以用一个哈希表 map 记录字符串 magazine 的中字符的数量，然后遍历字符串 ransomNote 的每个字符 ch，如果在 map 中不存在键 ch 或值为 0 则返回 false（表示 magazine 中字符不存在或数量不够），存在时则令值减一。成功遍历全部字符后返回 true。

优化：对于所有字符的范围可知时可以用一个 int 数组统计，速度更快。

40. 同构字符串。

方法：哈希表。根据题目可知，字符映射应该满足单射条件（即变量字符 x 和函数值字符 y 一一对应），所以用两个哈希表维护两个字符串中字符的映射关系（ a→b，b→a），key 保存变量值，value 记录函数值。依次遍历字符串的字符，如果两个哈希表都不存在其映射关系（都不存在 key），则添加新的映射关系到哈希表即可，如果只有其中一个哈希表存映射关系（存在 key），则说明字符集不满足单射关系，返回条件 false，如果两个哈希表都存在其 key，判断对应的值是否为对方即可，不是则不满足单射关系。全部遍历成功即满足单射关系。

优化：对于字符集可知时可以用长度为 128 的两个数组代替哈希表。

41. 单词规律

方法：哈希表。根据题目要求，pattern 的字符 ch 和 s 中的单词 word 满足单射关系，可以用两个哈希表分别维护字符 ch 到单词 word 的映射和单词 word 到字符 ch 的映射；单词 word 可以使用双指针方法即可找到。

值得注意的是：pattern 中的字符数可能和 s 中的单词数不一致。

42. 有效的字母异位词

方法：哈希表；字母异位词指的是改变字符串中单符的位置形成的新单词，所以字母异位词的字符种类和数量都不改变，所以先判断两个字符串的长度是否相等，所以可以用哈希表记录其中一个单词的字符的数量；依次判断另一个单词的字符是否都存在，存在就数量减一，如果数量扣减之后小于零，则代表不是异位词。

优化：对于字符范围可知时，可以采用数组代替哈希表。

43. 字母异位词分组

方法：哈希表+排序。字母异位词指改变字符位置得到的新单词称为原单词的异位词，故新单词的长度和各字符的数量不变；可以将遍历的单词的字符进行排序，字母异位词排序后应该是相同的单词；由于 Java 中字符串不可变，也可以采取统计字符串各字符的个数 Cnt，按从小到大的顺序构造单词，字母异位词构造的单词也应相同。
排序：char[] chars=str.toCharArray();Arrays.sort(chars);String key=new String(chars);
计数：利用 StringBuilder 对象构造字符串

44. 两数之和（无序数组）
方法：哈希表。由于数组无序，判断值 target-nums[i]是否在数组存在时需要遍历整个数组，导致整体复杂度达到 $O(n^2)$，所以可以用哈希表记录数组中元素的值和索引的映射，可以实现 O(1)判断一个值是否在数组中存在。流程：边遍历数组，判断 target-nums[i]是否存在，不存在就更新哈希表（把当前元素的值和索引添加到哈希表中），存在即找到结果。

45. 快乐数
方法：哈希表或快慢指针。根据题目我们能得到一些关键信息，根据计算规则得到的下一个数一定是大于 0 的，那么只可能存在三种情况：（1）不断计算之后得到 1；（2）不断计算进入循环；（3）不断计算达到无穷大。
我们首先分析第三种情况的可能性：

Digital	Max	Next
2	99	162
3	999	243
4	9999	324
...	...	...
11	99999999999	891（81*11）

我们发现当数位达到 4 或 4 以上，范围内的最大值计算下一个数远远比本身小（数位被极大的缩小），而只有数位为 3 时，其最大值的下一个数能保持数位为 3，也就是说经过不断计算后，数不断减小直到变为 1，那么最终进入循环圈（且圈内的数范围数在 243 以下，因为数位为 3 的最大数的下一个数为 243）。
所以问题转换为：判断数不断计算的过程中是否会进入循环圈。
判断是否存在循环圈的方法：方法一：哈希表。我们可以将计算过程中的数值用一个哈希表记录下来，当下一个数值已在哈希表存在时，说明存在循环圈。 方法二：采用快慢指针。用两个指针 fast 和 slow，fast 每次计算两步骤，slow 每次计算一步骤，如果存在循环，最终 fast 和 slow 会重合（超圈了），或者直到 fast 计算结果为 1.

46. 存在重复元素||
方法：哈希表。用哈希表记录数组中的元素的值和索引的映射关系，遍历的过程中判断哈希表中是否存在当前值（key），如果不存在就建立新映射关系，存在就判断旧值 PreIndex 是否满足要求（CurIndex-PreIndex<=k)，如果满足就返回 true，不满足就更新哈希表（用当前索引代替旧值）。 空间复杂度：O(n)
方法二：滑动窗口。也可以用哈希表维护固定长度 k 的滑动窗口，只要窗口内有重复值就返回 true，没有就移动窗口（删除旧值，增加新值）。空间复杂度：O(k)

47. 最长连续序列
方法：哈希表。我们很快能想到的方法是：假设当前遍历到的值为 num，依次判断数组中是

否存在 num+1，num+2，...，但判断数是否存在就需要 O(n)的平均复杂度，所以可以用哈希表记录数组中的值，就可以实现 O(1)判断是否值在数组中存在；但是我们发现当数组单调自增时（即 x,x+1,x+2,x+3,...），此时最坏情况仍然会达到 $O(n^2)$，所以我们可以换个思路，遍历到元素值为 x 时，我们先判断 x-1 是否在哈希表中存在，如果存在，说明 x 并不是所属连续序列的开头，直接进入下次外循环，不存在则说明 x 是连续序列的开头，我们进入内循环，依次判断 x+1，x+2，x+3...是否存在，同时更新结果。（优化：如果某个序列长度达到数组长度一半，则说明该序列为最长连续序列）

48. 汇总区间

方法：双指针。用两个指针 start=0 和 end=0 统计每个子区间的范围，循环判断如果 nums[end+1]等于 nums[end]+1，则 end++，循环退出时 start 和 end 分别指向子区间的起始索引和结束索引。

49. 合并区间

方法：排序。如果两个子区间表示的范围出现了相交，则可以进行合并；假设数组中的所有区间已经有序时，即对于子区间 $(x_1,y_1),(x_2,y_2),...,(x_n,y_n)$，有 $i<j$时，$x_i <= x_j; x_i = x_j$时，有$y_i <= y_j$；此时判断两个区间是否能合并：若 $x_2 <= y_1$时，此时两个子区间出现了相交，即可以进行合并；反之若 $x_2 > y_1$，则两个区间没有出现相交，无法合并，作为新的单独区间。